

# Object-Oriented Programming in Java

## Lecture 7 - Graphical User Interfaces

Emily Lucia Antosch

HAW Hamburg

03.10.2025

# Contents

|                                             |    |
|---------------------------------------------|----|
| 1. Introduction .....                       | 2  |
| 2. Basic Structure .....                    | 6  |
| 3. Creating Graphical User Interfaces ..... | 11 |
| 4. Layout .....                             | 19 |
| 5. Zeichnen .....                           | 25 |
| 6. Buttons & Events .....                   | 31 |
| 7. Simple Dialogs .....                     | 45 |
| 8. Suggestions .....                        | 48 |
| 9. License Notice .....                     | 50 |

# 1. Introduction

---

# 1.1 Where Are We Currently?

- The last lecture was about interfaces and abstract classes
- You can now
  - ▶ use abstract classes to structure your code more precisely,
  - ▶ implement interfaces to represent properties of classes,
  - ▶ assign classes and objects with an order using Comparable,
- Today we continue with **graphical user interfaces**.

# 1.1 Where Are We Currently?

## 1. Introduction

1. Imperative Concepts
2. Classes and Objects
3. Class Library
4. Inheritance
5. Interfaces
6. **Graphical User Interfaces**
7. Exception Handling
8. Input and Output
9. Multithreading (Parallel Computing)

# 1.2 The Goal of This Chapter

- You create graphical user interfaces with e.g. menus, buttons and text fields.
- You draw diagrams from simple geometric shapes (e.g. lines, circles).
- You respond to events (e.g. pressing a button) by connecting graphical elements with methods to be executed on user input.
- You use the Observer pattern so that objects of any data type can react to events.

## 2. Basic Structure

---

### ? Question

- What types of elements do you see?
- How do the elements react? Do elements interact with each other?

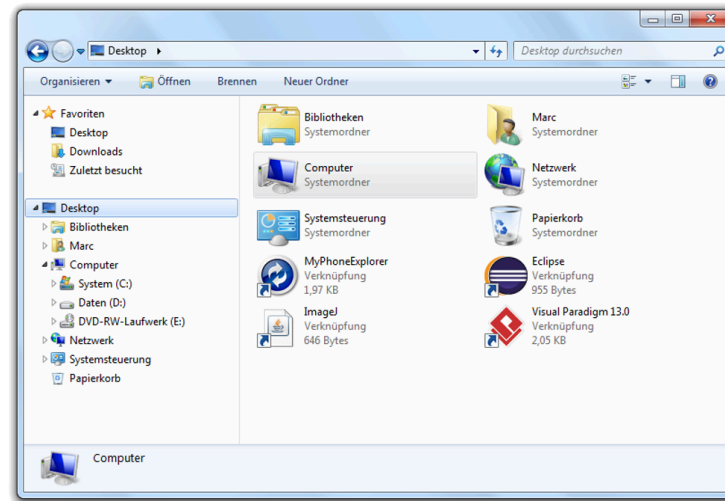


Figure 1: Windows 7 Explorer



## 2.1 Graphical UI

- Graphical user interface: Graphical user interface (GUI)
- Class libraries AWT and Swing already included in the Java SDK
- Abstract Window Toolkit (AWT):
  - ▶ Already introduced with Java 1.0
  - ▶ Only basic interface elements to support as many operating systems as possible (“Lowest common denominator”)
  - ▶ Uses the native elements (“widgets”) of the operating system
  - ▶ Originally full of design errors, as it was created under great pressure in just under two months
- Swing:
  - ▶ Extension of AWT
  - ▶ No more direct addressing of window functions of the current platform
  - ▶ Complete control over display elements

## 2.1 Graphical UI

- Base element: Frame
- Contains window bar with title and control elements (e.g. “Close”)
- Contains area where elements can be placed (Content pane)
- Can additionally contain menu bar

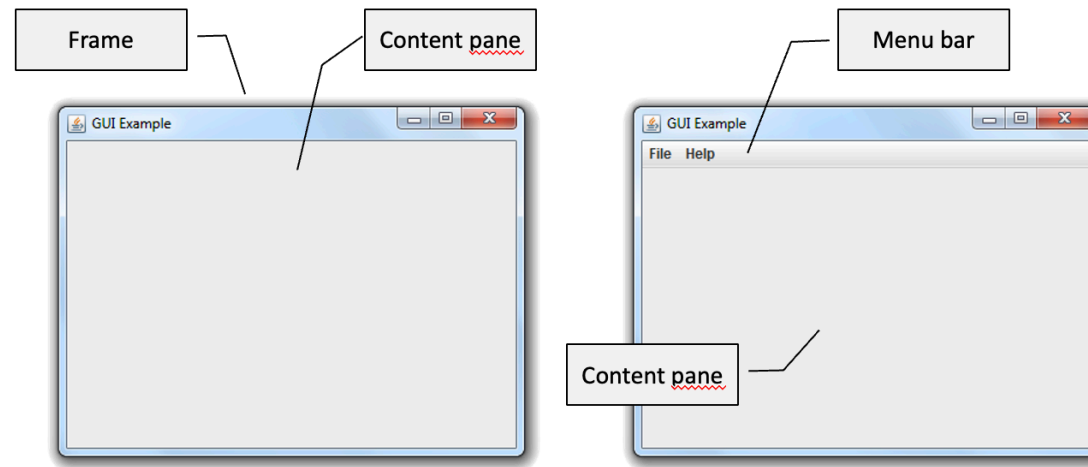


Figure 2: Structure of a frame

# 2.1 Graphical UI

## 2. Basic Structure

- Elements are added hierarchically.
- For elements that contain other elements, the layout can be specified.

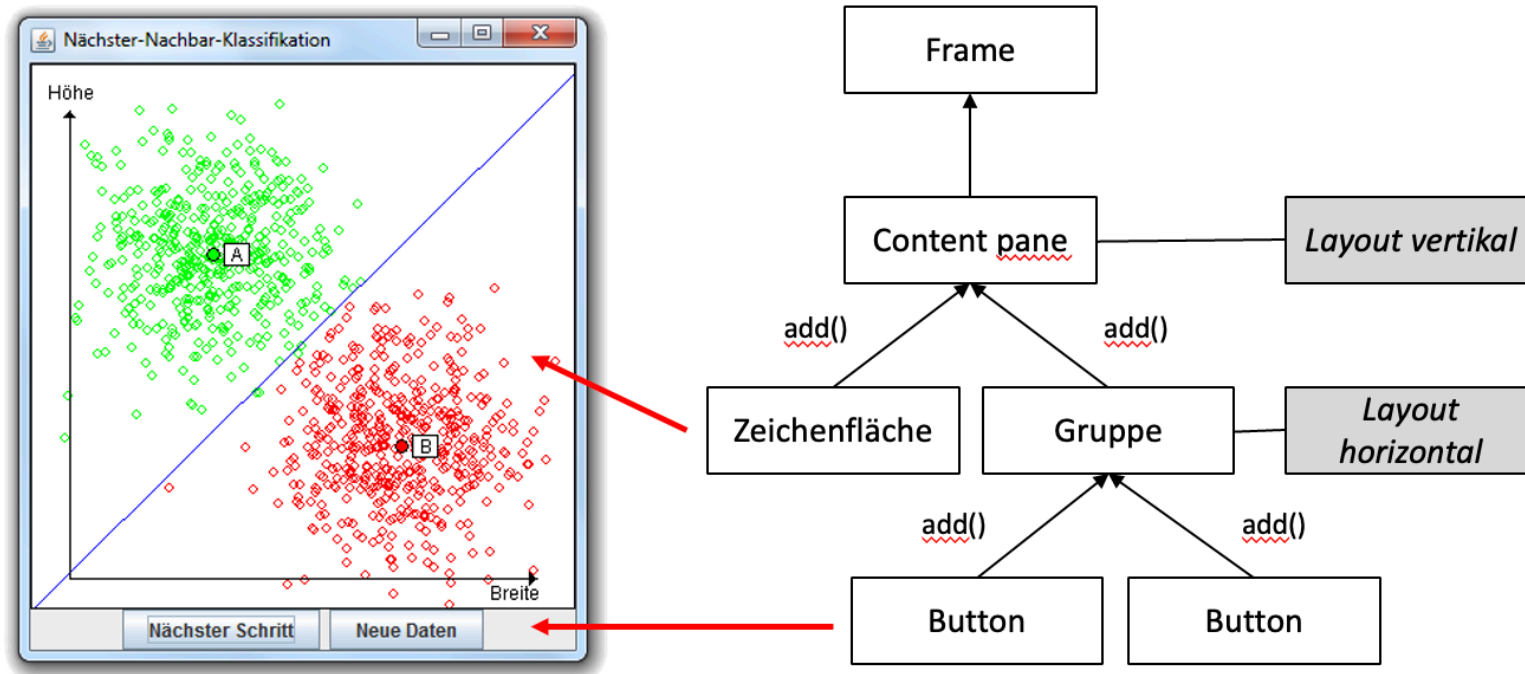


Figure 3: Hierarchy of a window

# **3. Creating Graphical User Interfaces**

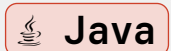
---

## 3.1 Simple Program

## 3. Creating Graphical User Interfaces

- Executable main() method creates object of the class
- Class creates frame with graphical interface in constructor
- Specify “Close Operation” so that application terminates when window is closed

```
1  public class HelloWorld {
2      public HelloWorld() {
3          JFrame frame = new JFrame("GUI example");
4          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
5          frame.setVisible(true);
6      }
7
8      public static void main(String[] args) {
9          new HelloWorld();
10     }
11 }
```



### ? Question

- What happens if the “Close Operation” is not set to “Exit on close”?
  - Why do you have to explicitly display the window via `setVisible(true)`?
- 
- And it doesn't look really nice:
    - ▶ The window is too small!
    - ▶ The window “sticks” in the upper left corner!



### ☰ Task 1

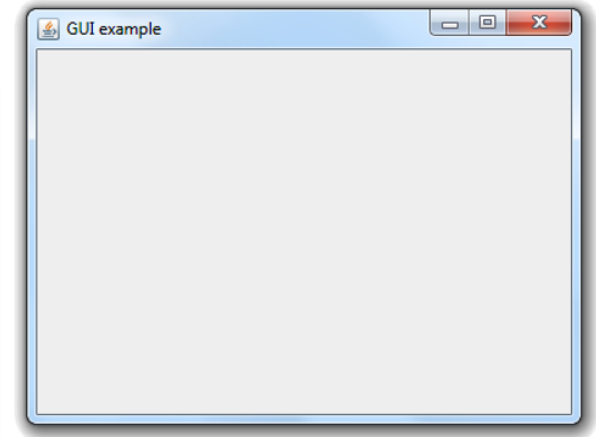
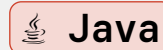
- Enlarge it to 400 x 300 pixels (width x height).
- Place it 50 pixels from the left and top edge respectively.
- Hint: Display the methods of `frame`.

# 3.1 Simple Program

## 3. Creating Graphical User Interfaces

- Corrected size and position:

```
1  public class HelloWorld {
2      public HelloWorld() {
3          JFrame frame = new JFrame("GUI example");
4
5          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6          frame.setSize(400, 300);
7          frame.setLocation(50, 50);
8          frame.setVisible(true);
9      }
10
11     public static void main(String[] args) {
12         new HelloWorld();
13     }
```



# 3.1 Simple Program

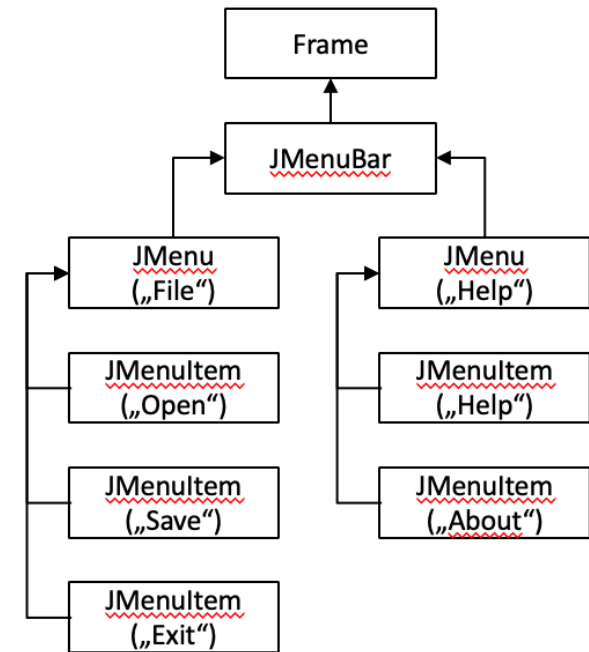
## 3. Creating Graphical User Interfaces

- Klassen:
  - ▶ JMenuBar: Menu bar
  - ▶ JMenu: Menu in menu bar (e.g. File, Help)
  - ▶ JMenuItem: Entry in a menu (e.g. New, Save as)
- Add the following menus to our program:
  - ▶ Menu File with menu items Open, Save and Exit
  - ▶ Menu Help with menu items Help and About



### Tip

- Elements are usually added via add().
- The menu bar is added via setJMenuBar().



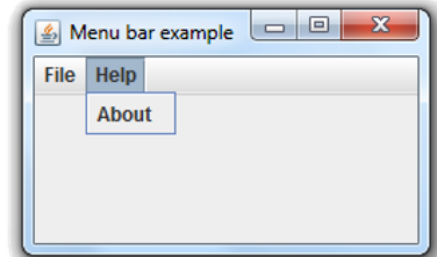
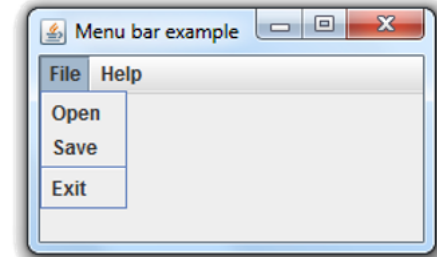


# 3.1 Simple Program

## 3. Creating Graphical User Interfaces

```
1  public MenuBar() {
2      JFrame frame = new JFrame("Menu bar example");
3      // Set frame properties ...
4
5      JMenuBar menuBar = new JMenuBar(); // Create menu bar and add to frame
6      frame.setJMenuBar(menuBar);
7
8      JMenu menuFile = new JMenu("File"); // Create menu "File"
9      menuBar.add(menuFile);
10     menuFile.add(new JMenuItem("Open"));
11     menuFile.add(new JMenuItem("Save"));
12     menuFile.addSeparator();
13     menuFile.add(new JMenuItem("Exit"));
14
15     JMenu menuHelp = new JMenu("Help"); // Create menu "Help"
16     menuBar.add(menuHelp);
17     menuHelp.add(new JMenuItem("About"));
18
19     frame.setVisible(true);
20 }
```

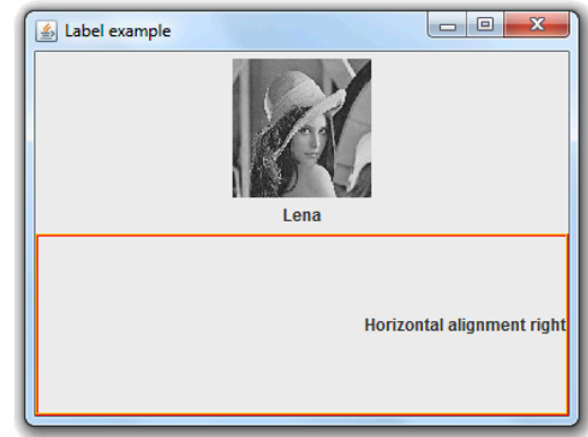
Java



## 3.1 Simple Program

## 3. Creating Graphical User Interfaces

- Class JLabel displays non-editable text
  - ▶ Can be aligned horizontally and vertically (e.g. centered)
  - ▶ Can draw borders
  - ▶ Can also display images
- Let's create the window shown on the right:
  - ▶ Load image via new ImageIcon()
  - ▶ Border via BorderFactory.createEtchedBorder()
  - ▶ Add label to content pane via add()
  - ▶ Layout via frame.setLayout(new GridLayout(2, 1))

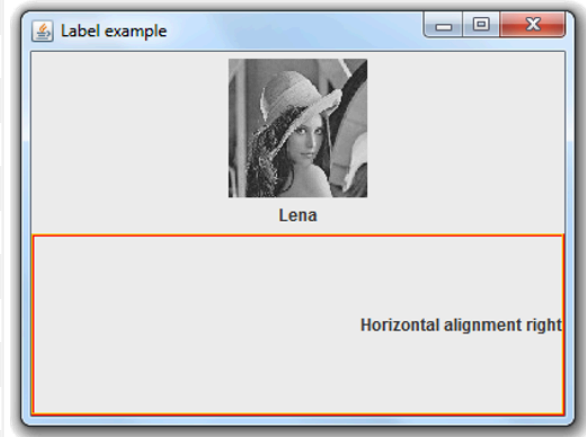


# 3.1 Simple Program

## 3. Creating Graphical User Interfaces

```
1 // Create frame and set properties
2 JFrame frame = new JFrame("Label example");
3 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
4 frame.setSize(400, 300);
5 frame.setLocation(50, 50);
6 frame.setLayout(new GridLayout(2, 1)); // 2 rows, 1 column
7
8 // Create labels
9 ImageIcon image = new ImageIcon("folien07_gui/Lena100.jpg");
10 JLabel label1 = new JLabel("Lena", image, JLabel.CENTER);
11 label1.setHorizontalTextPosition(JLabel.CENTER);
12 label1.setVerticalTextPosition(JLabel.BOTTOM);
13
14 JLabel label2 = new JLabel("Horizontal alignment right");
15 label2.setHorizontalAlignment(JLabel.RIGHT);
16 label2.setBorder(BorderFactory.createEtchedBorder(Color.RED, Color.ORANGE));
17
18 // Add labels to content pane
19 Container contentPane = frame.getContentPane();
20 contentPane.add(label1);
21 contentPane.add(label2);
22
23 frame.setVisible(true);
```

 Java



## 4. Layout

---

## 4.1 Layout-Manager

- Define the arrangement of GUI elements
- Various layout managers defined, e.g.:
  - ▶ BorderLayout:
    - Elements on top of each other (“vertical”) or next to each other (“horizontal”)
  - ▶ GridLayout:
    - Elements placed in uniform grid
    - All cells have the same size
  - ▶ FlowLayout:
    - Elements placed in row like horizontal BorderLayout
    - However, line break as soon as a line is “full”

# 4.1 Layout-Manager

## 4. Layout

```
1 // Create frame and set properties
2 JFrame frame = new JFrame("Layout example");
3 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
4 frame.setLocation(50, 50);
5
6 // Create contents
7 Container contentPane = frame.getContentPane();
8 contentPane.setLayout(new BoxLayout(contentPane,
9   BoxLayout.Y_AXIS));
10 contentPane.add(new JButton("Ready"));
11 contentPane.add(new JButton("Set"));
12 contentPane.add(new JButton("Go"));
13 contentPane.add(new JButton("los!"));
14
15 frame.pack();
16 frame.setVisible(true);
```

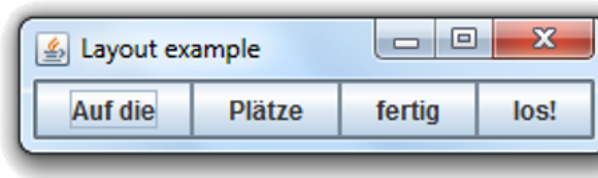
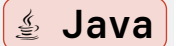


# 4.1 Layout-Manager

## 4. Layout

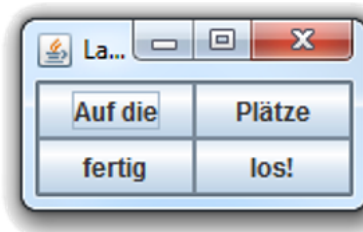
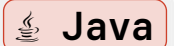
- Horizontal BoxLayout:

```
1 contentPane.setLayout(new BorderLayout(contentPane, BorderLayout.X_AXIS));
```



- GridLayout:

```
1 contentPane.setLayout(new GridLayout(2, 2));
```

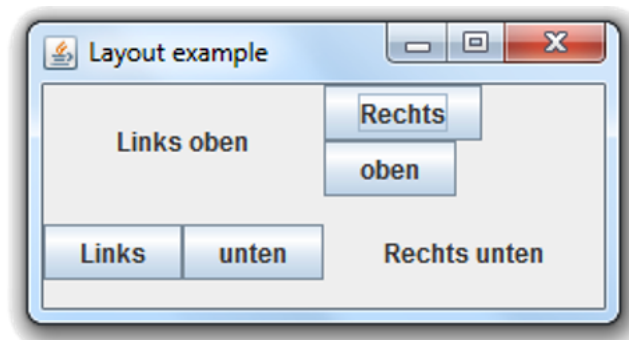


## 4.1 Layout-Manager

- Elements can be grouped in objects of class JPanel.
- Each JPanel object has its own layout manager.

### ? Question

- ▶ Which elements does the shown window contain?
- ▶ Via which objects and layout managers are these arranged?

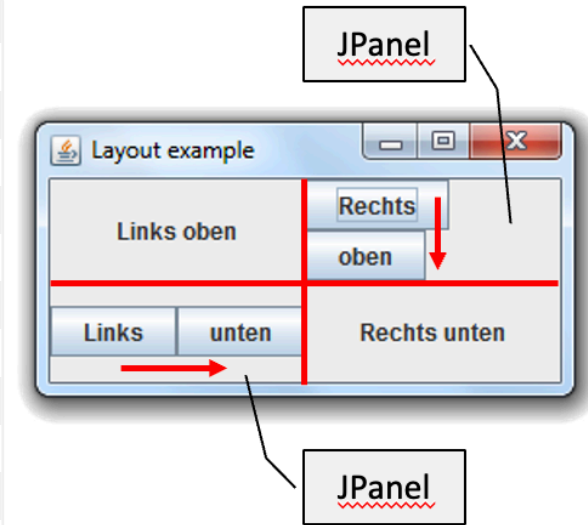




# 4.1 Layout-Manager

## 4. Layout

```
1  JFrame frame = new JFrame("Layout example");
2
3  JPanel panel1 = new JPanel();
4  panel1.setLayout(new BorderLayout(panel1, BorderLayout.Y_AXIS));
5  panel1.add(new JButton("Right"));
6  panel1.add(new JButton("top"));
7
8  JPanel panel2 = new JPanel();
9  panel2.setLayout(new BorderLayout(panel2, BorderLayout.X_AXIS));
10 panel2.add(new JButton("Left"));
11 panel2.add(new JButton("bottom"));
12
13 Container contentPane = frame.getContentPane();
14 contentPane.setLayout(new GridLayout(2, 2));
15 contentPane.add(new JLabel("Top left", JLabel.CENTER));
16 contentPane.add(panel1);
17 contentPane.add(panel2);
18 contentPane.add(new JLabel("Bottom right", JLabel.CENTER));
19
20 frame.pack();
21 frame.setVisible(true);
```



## 5. Zeichnen

---

# 5.1 Zeichnen

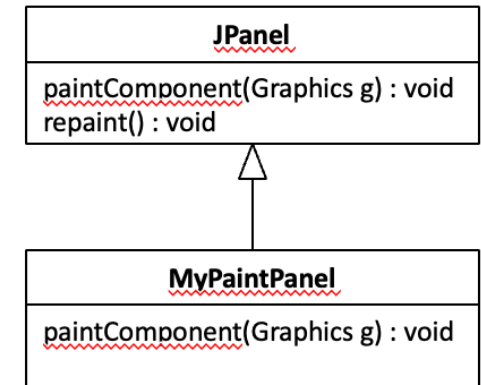
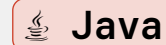
- Class JPanel as drawing surface:
  - ▶ Can draw freely on panel.
- Drawing method:
  - ▶ System executes paintComponent() method for drawing
  - ▶ Is automatically called when window changes
  - ▶ Method receives parameter of type Graphics (graphics context)
  - ▶ Graphics has methods for drawing (texts, lines, rectangles, arcs, ...)
- Explicit redrawing:
  - ▶ Redrawing can also be initiated via repaint() method.
  - ▶ This internally calls paintComponent().

| JPanel                                                                |
|-----------------------------------------------------------------------|
| <u>paintComponent</u> (Graphics g) : void<br><u>repaint</u> () : void |

# 5.1 Zeichnen

- Okay, so there is JPanel with the `paintComponent()` method.
- What is drawn is what is in `paintComponent()`.
- But how can you add drawing commands to this method?!
- Solution:
  - ▶ Derive from `JPanel` and override `paintComponent()`.
  - ▶ This results in: Panel class with freely definable drawing method

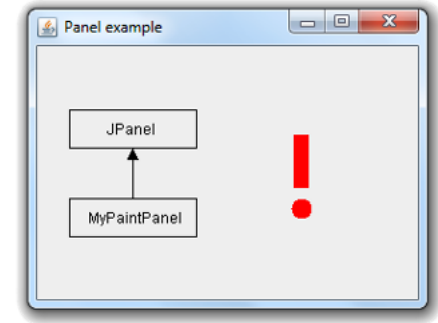
```
1  class MyPaintPanel extends JPanel {  
2      public void paintComponent(Graphics g) {  
3          super.paintComponent(g);  
4          // Code for own drawings ...  
5      }  
6  }
```



# 5.1 Zeichnen

## 5. Zeichnen

```
1  class MyPaintPanel extends JPanel {
2      public Dimension getPreferredSize() {
3          return new Dimension(300, 200);
4      }
5
6      public void paintComponent(Graphics g) {
7          super.paintComponent(g);
8
9          g.setColor(Color.BLACK);
10         g.drawRect(25, 50, 100, 30); // Super class
11         g.drawString("JPanel", 55, 70);
12         g.drawRect(25, 120, 100, 30); // Sub class
13         g.drawString("MyPaintPanel", 40, 140);
14         g.drawLine(75, 80, 75, 120); // Arrow
15         g.fillPolygon(new int[]{70, 75, 80}, new int[]{90, 80, 90}, 3);
16
17         g.setColor(Color.RED);
18         g.fillRect(202, 70, 12, 42);
19         g.fillOval(200, 120, 16, 16);
20     }
21 }
```

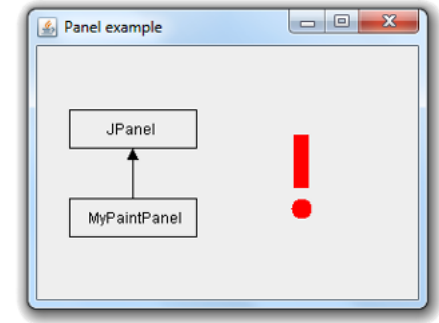
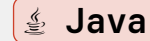


# 5.1 Zeichnen

## 5. Zeichnen

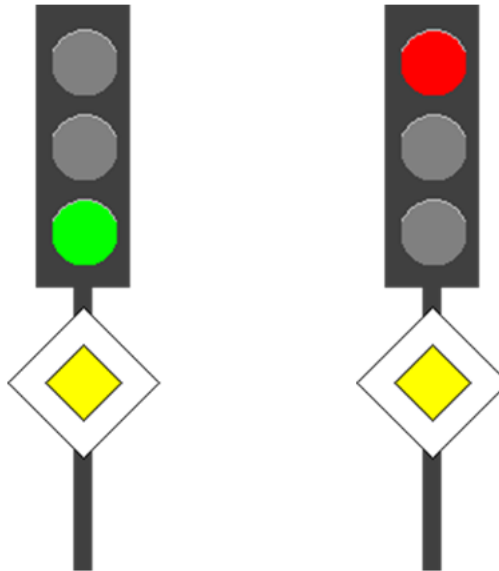
- Integration into graphical interface:

```
1  public class PaintPanel {
2      public PaintPanel() {
3          JFrame frame = new JFrame("Panel example");
4          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
5          frame.setLocation(50, 50);
6
7          frame.add(new MyPaintPanel());
8          frame.pack();
9          frame.setVisible(true);
10     }
11
12     public static void main(String[] args) {
13         new PaintPanel();
14     }
15 }
```



### ☰ Task 2

- Discover your artistic streak!
- Create a program that displays a traffic light.



## 6. Buttons & Events

---



# 6.1 Task

## 6. Buttons & Events

- Our goal is the following application:
  - ▶ Window with three buttons and one panel
  - ▶ Selection of buttons color the panel red, blue or in random color

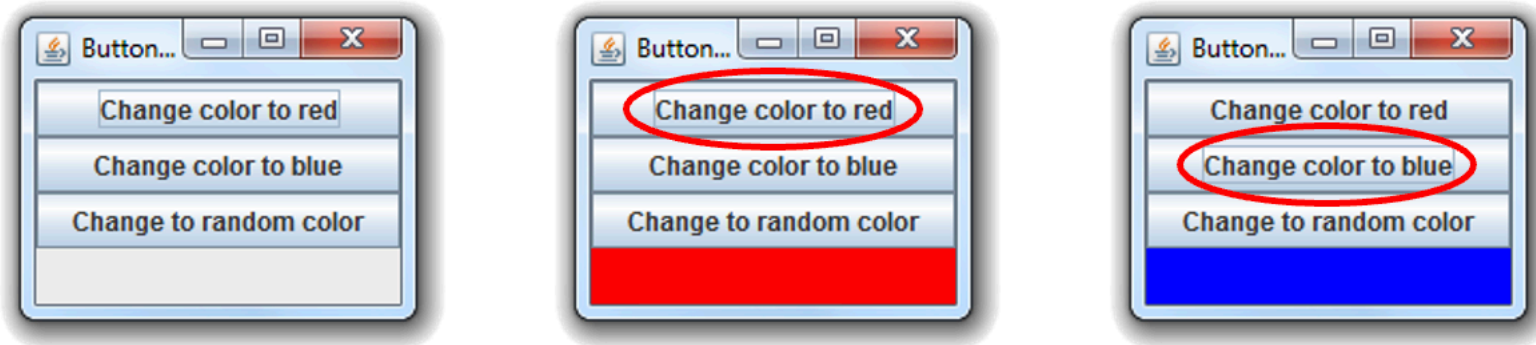
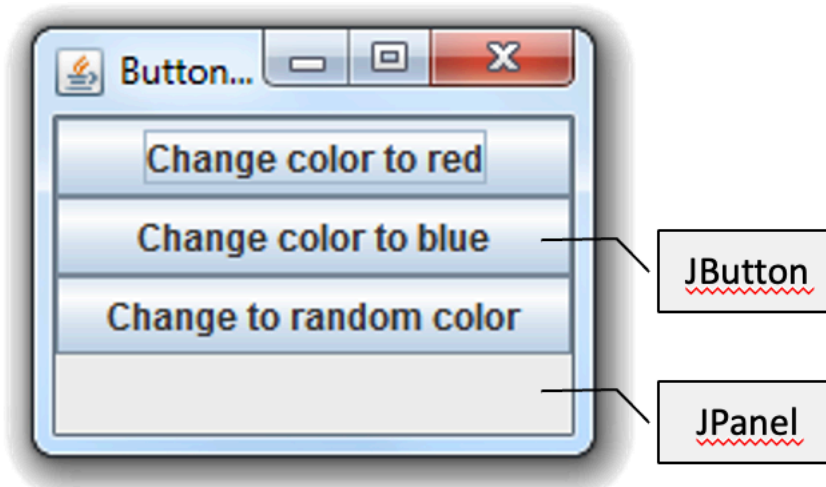


Figure 21: Buttons that change a color

- We need for this:
  - ▶ Buttons as elements
  - ▶ Possibility to react to pressed button

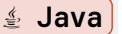
### Task 3

- First create the GUI with its elements.



- Creating elements of class JButton:

```
1  public class ButtonEvent {
2      public ButtonEvent() {
3          JFrame frame = new JFrame("Button example");
4          // Set frame properties ...
5
6          // Create and layout contents
7          frame.setLayout(new GridLayout(4, 1)); // 4 rows, 1 column
8          Container contentPane = frame.getContentPane();
9          contentPane.add(new JButton("Change color to red"));
10         contentPane.add(new JButton("Change color to blue"));
11         contentPane.add(new JButton("Change to random color"));
12         contentPane.add(new JPanel());
13         frame.pack();
14         frame.setVisible(true);
15     }
16
17     public static void main(String[] args) {
18         new ButtonEvent();
19     }
20 }
```



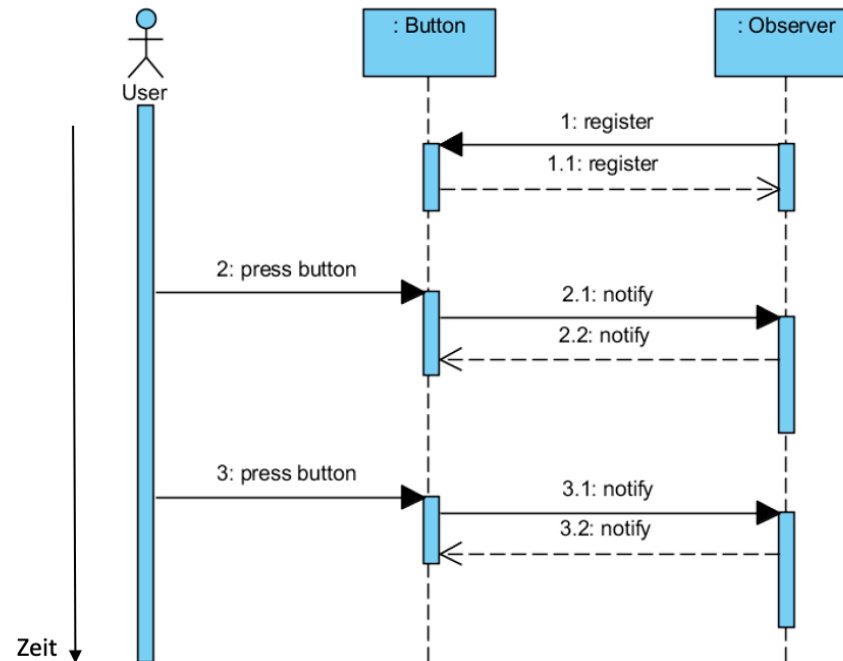
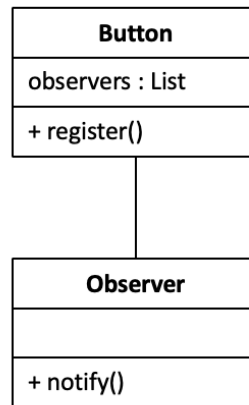
## 6.1 Task

- But how can we react when a button is pressed?
- Involved objects:
  - ▶ Button with state (e.g. “not pressed”, “pressed”)
  - ▶ Object that should be notified when the button changes
- Basic approach:
  - ▶ Registration:
    - Object “tells the button” that it wants to be notified of changes
    - Button remembers (e.g. in list) which objects should be notified
  - ▶ Button is pressed:
    - Button notifies objects in the list that its state has changed

# 6.1 Task

## 6. Buttons & Events

- Possible implementation:
  - ▶ Button: Method register() to add observers to the list
  - ▶ Observer: Method notify() that button object calls for notification



- Buuuuuut:
  - ▶ JButton cannot know classes we created.
  - ▶ Therefore cannot know if we implemented method notify().
- Solution:
  - ▶ Observers implement a defined interface
  - ▶ Button doesn't need to know the observer's class, only the interface

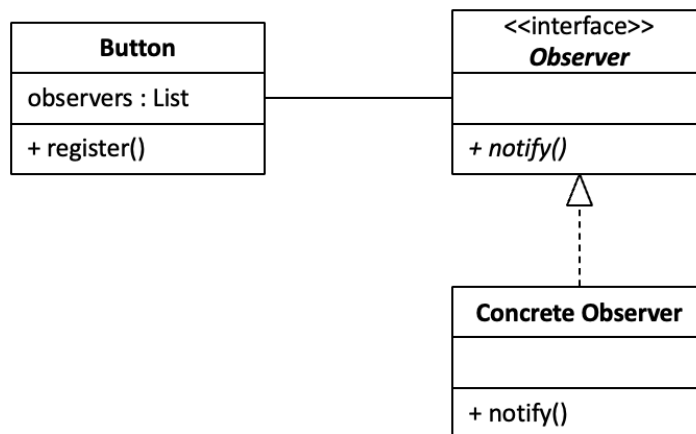


Figure 24: Interface Observer

## 6.1 Task

- Approach is also called Observer pattern
- More than one observer can register.
- In Swing, names of interface and methods chosen differently:

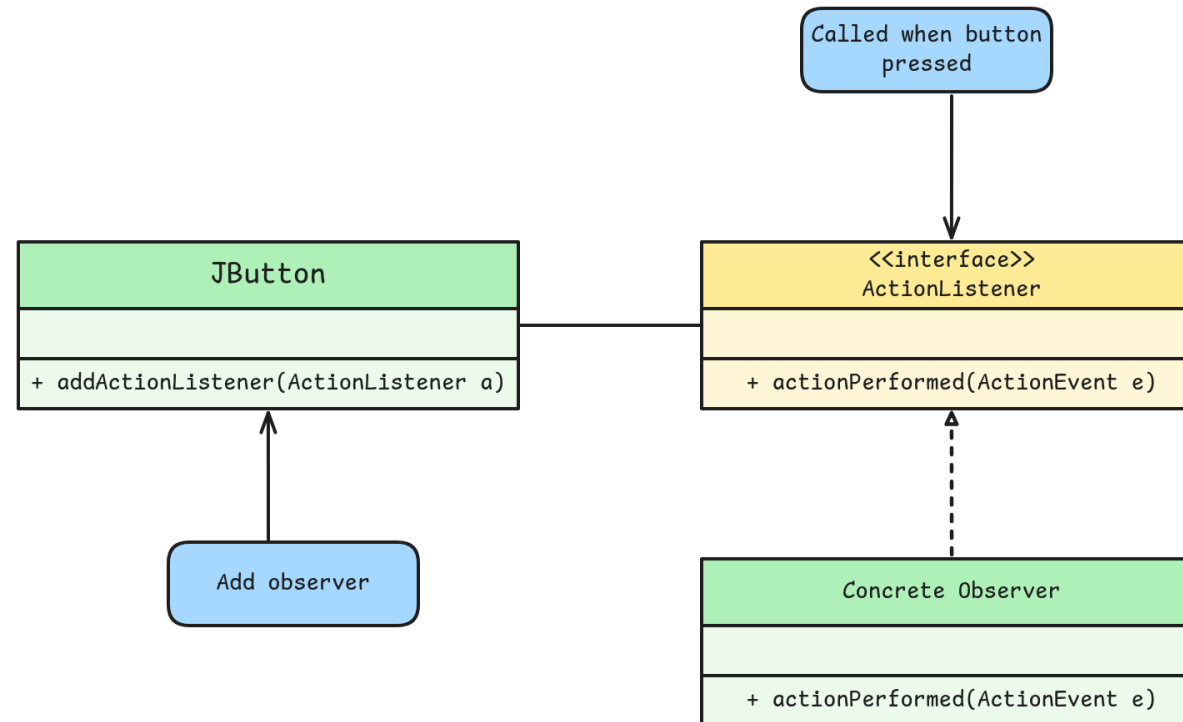
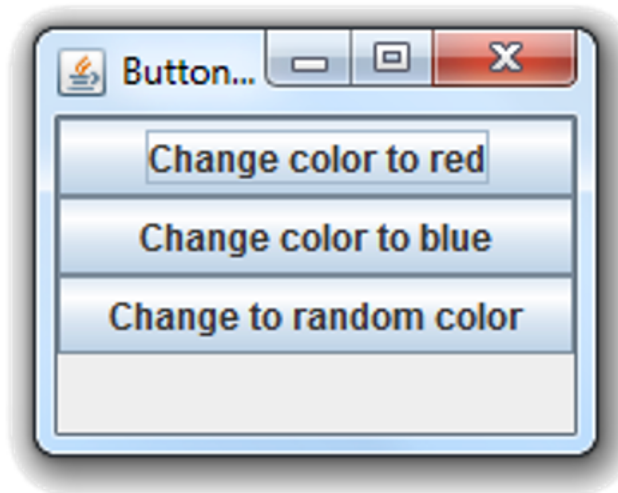


Figure 25: Observer pattern

### Task 4

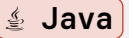
- Executable class implements interface ActionListener
- Object of executable class registers itself with the buttons





- Excerpt from source code:

```
1  public class ButtonEvent implements ActionListener {
2      private JPanel panel;
3      private JButton buttonRed, buttonBlue, buttonRandom;
4
5      public ButtonEvent() {
6          // ...
7
8          // Buttons with event handling
9          buttonRed = new JButton("Change color to red");
10         buttonBlue = new JButton("Change color to blue");
11         buttonRandom = new JButton("Change to random color");
12
13         buttonRed.addActionListener(this);
14         buttonBlue.addActionListener(this);
15         buttonRandom.addActionListener(this);
16         // ...
17     }
18 }
```



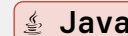
- Reaction to events (excerpt from source code):
  - ▶ Button identified via getSource() method of event object

```
1  public class ButtonEvent implements ActionListener {
2      public void actionPerformed(ActionEvent event) {
3          if (event.getSource() == buttonRed) {
4              panel.setBackground(Color.RED);
5          } else if (event.getSource() == buttonBlue) {
6              panel.setBackground(Color.BLUE);
7          } else if (event.getSource() == buttonRandom) {
8              Random random = new Random();
9              float red = random.nextFloat();
10             float green = random.nextFloat();
11             float blue = random.nextFloat();
12             Color color = new Color(red, green, blue);
13             panel.setBackground(color);
14         }
15     }
16 }
```



- Alternatively (find out in actionPerformed() which button was pressed):
- Connect buttons with a string, e.g.:

```
1 buttonRed.setActionCommand("Change color to red");
2 buttonBlue.setActionCommand("Change color to blue");
3 buttonRandom.setActionCommand("Change to random color");
```



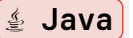
- Query and use string in actionPerformed() method:

```
1 public void actionPerformed(ActionEvent event) {
2     String actionCommand = event.getActionCommand();
3
4     if (actionCommand.equals("Change color to red")) {
5         // ...
6     } else if (actionCommand.equals("Change color to blue")) {
7         // ...
8     } else if (actionCommand.equals("Change to random color")) {
9         // ...
10    }
11 }
```



- Define new ActionListener with actionPerformed() method inline

```
1  public class ButtonEvent2 {
2      private JPanel panel;
3
4      public ButtonEvent2() {
5          // ...
6          buttonRed.addActionListener(new ActionListener() {
7              public void actionPerformed(ActionEvent event) {
8                  panel.setBackground(Color.RED);
9              }
10         });
11
12         // ...
13     }
14
15     public static void main(String[] args) {
16         new ButtonEvent2();
17     }
18 }
```



- All Swing components can register the following observers:
  - ▶ Component listener: Changes in size, position or visibility
  - ▶ Focus listener: Component gains or loses keyboard focus
  - ▶ Key listener: Keyboard events (only when component has keyboard focus)
  - ▶ Mouse listener: Mouse clicks, pressing, releasing and mouse movements
  - ▶ Mouse motion listener: Changes in cursor position over the component
  - ▶ Mouse wheel listener: Changes of mouse wheel over the component

## 7. Simple Dialogs

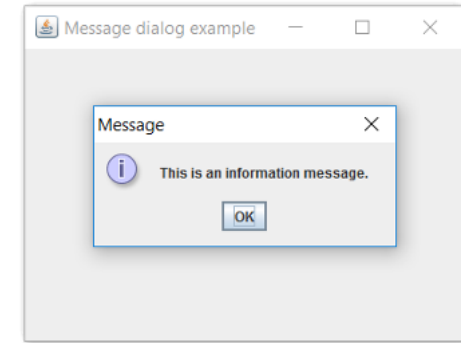
---

# 7.1 Simple Dialogs

- Examples for dialogs via JOptionPane:

```
1  public class MessageDialogs {
2      public MessageDialogs() {
3          // Create and show frame
4          JFrame frame = new JFrame("Message dialog example");
5          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
6          frame.setSize(400, 300);
7          frame.setLocationByPlatform(true);
8          frame.setVisible(true);
9
10         // Display dialogs
11         JOptionPane.showMessageDialog(frame, "This is a plain message.",
12             "Message",
13             JOptionPane.PLAIN_MESSAGE);
14         JOptionPane.showMessageDialog(frame, "This is an information
15             message.", "Message",
16             JOptionPane.INFORMATION_MESSAGE);
17         JOptionPane.showMessageDialog(frame, "This is a warning.", "Message",
18             JOptionPane.WARNING_MESSAGE);
19     }
20 }
```

## 7. Simple Dialogs



# 7.1 Simple Dialogs

## 7. Simple Dialogs

```
19     public static void main(String[] args) {  
20         new MessageDialogs();  
21     }  
22 }
```



## 8. Suggestions

---

## 8.1 Ideas for Experimenting

- Some additional GUI elements:
  - ▶ Text fields via `JTextField`, `JPasswordField` and `JTextArea`
  - ▶ Selection boxes via `JCheckBox`
  - ▶ Lists via `JComboBox` and `JList`
  - ▶ Tooltips via method `setToolTipText()`
  - ▶ File selection via `JFileChooser`

## 9. License Notice

---

## 9.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- <https://creativecommons.org/licenses/by-nc-sa/4.0/>
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.